

Lustre

An AI-powered jewellery product-photo editor that uses Google Gemini to replace plain backgrounds with editorial-quality environments

Project Status: Functional — Personal Project Date: April–June 2026

Summary

A full-stack web application — Express backend, React frontend — that sends jewellery product photographs to Google's Gemini image-generation API and returns background-enhanced versions suitable for e-commerce listings. The application supports iterative multi-turn refinement, eight named style presets, a multi-file upload queue, and a Canvas-based local compositor as a fallback when API quota is exhausted. Security headers required by the project's PRD are implemented.

The Problem

E-commerce jewellery sellers typically photograph products against plain white backgrounds — practical for speed but visually flat for premium listings. Professional editorial retouching (marble surfaces, silk folds, atmospheric lighting) is expensive and slow. Small sellers need a tool that can produce catalog-quality background enhancements quickly, without design expertise, and without compromising the visual accuracy of the jewellery itself.

Objective

Build a web application that: (1) accepts raw jewellery product photographs; (2) applies a user-selected style preset and aspect ratio; (3) sends the image to Gemini with a fidelity-preserving system prompt; (4) returns an editorial background-replaced PNG; (5) supports iterative refinement through conversational follow-up prompts using the latest generated image as context; and (6) degrades gracefully to a local Canvas compositor when the Gemini API is unavailable.

Intended Users

Jewellery e-commerce sellers — particularly small or independent sellers who cannot afford professional photography retouching for every product. The application is currently configured for single-user, localhost use; no authentication or multi-user support is present.

Why This Matters

Lustre illustrates a practical application of generative AI in an existing commercial workflow — product photography preparation — rather than a speculative use case. The project demonstrates several implementation decisions of broader relevance: the multi-turn image refinement pattern (passing the latest generated image, not the original, as the primary input on subsequent turns) produces markedly more coherent iterative results and is an underused capability of Gemini's vision API. The provider abstraction layer (AllImageProvider interface) allows switching between Google AI Studio and Vertex AI without

modifying application code. The Canvas compositor fallback converts a hard failure (API quota exhaustion) into a graceful degradation — maintaining user experience without requiring a second paid service.

Architecture and End-to-End Workflow

Single-origin Express + Vite application:

Browser

React 19 SPA (Tailwind v4, Vite HMR in dev)

App.tsx – state, upload, generate

└─ Sidebar.tsx

upload · presets · aspect ratio

model picker · queue badge

└─ EditorArea.tsx

canvas viewport · version tabs

└─ BeforeAfterSlider.tsx

POST /api/edit-image (JSON, base64 images)

Express (port 3000) ← server.ts

Security middleware: X-Content-Type-Options

X-Frame-Options, Cache-Control: no-store

getProvider() – reads AI_PROVIDER env var

└─ GoogleAISTudioProvider (GEMINI_API_KEY)

└─ VertexAIProvider (ADC / GCP project)

@google/genai v1.29.0 → Gemini generateContent()

responseModalities: ["TEXT", "IMAGE"]

Response: { imageUrl (PNG base64), feedback, promptUsed }

Multi-turn flow:

Turn 1: send original image + system prompt + preset

Turn 2+: send [latest generated image, original image]

+ history[] + new instruction

→ Gemini uses latest as primary editing base

Fallback:

HTTP 429 / quota error → compositor.ts (Canvas2D)

Each preset has a bespoke gradient/bezier backdrop

CSS multiply blending removes white photo background

Technologies Used

Technology	Version	Role
TypeScript	~5.8.2	End-to-end type safety (server + client)
React	19.0.1	Frontend SPA
Express	4.21.2	Backend API server
@google/genai	1.29.0	Gemini API SDK (image generation)
Vite	6.2.3	Dev server middleware + production build
Tailwind CSS	4.1.14	Utility-first styling (v4 via Vite plugin)
tsx	4.21.0	Runs server.ts directly in development
esbuild	0.25.0	Bundles server.ts for production
lucide-react	0.546.0	Icon library
dotenv	17.2.3	Environment variable loading

What Was Actually Built

Each capability is independently labelled based on evidence found in the repository.

Capability	Status
Gemini API image generation (responseModalities: TEXT + IMAGE)	Implemented
Multi-turn iterative refinement (latest generated as primary input)	Implemented
Eight named style presets with detailed Gemini prompts	Implemented
Five aspect ratios (1:1, 4:5, 16:9, 3:4, 4:3)	Implemented
Multi-file upload queue with badge + Load Next button	Implemented
Drag-and-drop file upload	Implemented
Before/After drag comparison slider	Implemented
Version history timeline (all turns in session)	Implemented
PNG export / download	Implemented
Local Canvas compositor (bespoke gradients for all 8 presets)	Implemented
Automatic 429/quota failover to Canvas compositor	Implemented
Product fidelity guardrail (system prompt, 3 rules)	Implemented
Privacy notice modal on first upload (Gemini data use)	Implemented
Security headers: X-Content-Type-Options, X-Frame-Options	Implemented
Cache-Control: no-store on all /api/edit-image response paths	Implemented
Provider abstraction layer (AllImageProvider interface)	Implemented
Google AI Studio provider (GEMINI_API_KEY)	Implemented
Vertex AI provider (Application Default Credentials)	Implemented
Model selector: 2.5 Flash default	Implemented
Model selector: 3.1 High-Q button (gemini-3.1-flash-image)	Implemented with limitations
Automated tests (unit / integration)	Not present
Authentication / authorisation	Not present
Rate limiting on /api/edit-image	Not present
Persistent session storage (versions lost on refresh)	Not present
CORS configuration	Not present

Implementation Notes

All twelve commits are authored by 'neversettledwanderer' (the user). Development took place across two main sessions: 27 April 2026 (initial implementation, model fix) and May–June 2026 (security, UX polish, provider abstraction, Vertex AI). There is no evidence of a starter template or generated scaffold; the codebase appears to be an original implementation.

The Gemini API call passes two images on turns 2 and above: the latest generated PNG as the first (primary) inlineData part and the original raw upload as the second (reference) part. This ordering is explicit in the prompt text and ensures that Gemini refines its own previous output rather than re-editing from the original photograph — producing significantly more coherent iterative results.

The Canvas compositor provides eight preset-specific implementations using CSS 'multiply' blending mode, which removes the white photo background without a segmentation mask. Each preset renders bespoke elements: the 'Etsy Listing' preset draws oak-wood plank grain using Bezier curves; 'Bridal Elegance' renders ellipse bokeh circles; 'Festive Sparkle' adds 18 randomised bokeh arcs (producing non-deterministic output per run).

The 'gemini-3.1-flash-image' model ID used in the second model-selector button may not correspond to a currently available Gemini model. Commit 9 ('fix: use image models actually available to the API key') addressed the previous model ID issue; the 3.1 button was added in the same session but its validity is not independently confirmed by the repository evidence.

User Contribution and Role

All twelve commits in the repository are authored by the user (GitHub handle: neversettledwanderer). The user wrote the full-stack implementation: Express backend, React SPA, Gemini API integration, Canvas compositor, provider abstraction, security middleware, and all component logic. No AI code-generation platform or starter template was identified.

Testing and Validation Evidence

There are no test files, test directories, or testing frameworks in the repository. The package.json 'lint' script runs 'tsc --noEmit' (TypeScript type-checking only). The implementation plan (docs/implementation-plan.md) describes manual verification steps for each feature but records no pass/fail results. The project's PRD (lustre_prd.pdf) includes acceptance criteria but no evidence of formal test execution was found.

Known Limitations

- No automated tests — the only verification is TypeScript type-checking (tsc --noEmit).
- No authentication — the /api/edit-image endpoint is open; anyone with network access can use the GEMINI_API_KEY.
- No rate limiting — a client can send unlimited requests, exhausting the API quota rapidly.
- In-memory state only — version history and upload queue are held in React state; a page refresh loses all work.
- The 'gemini-3.1-flash-image' model ID in the second selector button is of unverified availability.
- No CORS configuration — fine for same-origin localhost use; would block external API consumers.
- Images are transferred as base64 JSON (up to 50 MB limit); multipart form upload would be more efficient for large files.
- The 'motion' package (^12.23.24) is declared in dependencies but not imported in any component — a dead dependency.

- No CHANGELOG or .env.example file found in the working tree (referenced in commit messages and implementation plan).

Lessons Learned

The multi-turn image refinement pattern — passing the latest generated image as the primary input, not the original photograph — is the key insight of this project. It converts a single-pass generation tool into a genuine iterative editor, allowing users to add warmth, adjust reflections, or change surface materials across successive turns without losing continuity. The Canvas compositor fallback demonstrates that graceful degradation from a paid AI API to a deterministic local alternative is both technically achievable and worth implementing — it removes the risk of a completely broken experience when quota runs out. The provider abstraction layer (AllImageProvider interface) anticipates future migration to Vertex AI for GCP-credit-eligible billing without requiring changes to application logic.

Practical Next Steps

1. Verify the 'gemini-3.1-flash-image' model ID with a live ListModels call; replace or remove if unavailable.
2. Add express-rate-limit middleware to /api/edit-image before any public deployment.
3. Write integration tests for the /api/edit-image endpoint (at minimum: valid image returns PNG; missing image returns 400; missing API key returns 401).
4. Replace the in-memory version history with sessionStorage or IndexedDB to survive page refresh.
5. Add a .env.example file documenting all required environment variables (GEMINI_API_KEY, AI_PROVIDER, GOOGLE_CLOUD_PROJECT, GOOGLE_CLOUD_LOCATION).
6. Audit and remove the unused 'motion' package dependency.
7. Add CORS configuration if the server is ever exposed beyond localhost.

Evidence Base Reviewed

- server.ts (full read — Express setup, security middleware, /api/edit-image handler)
- src/App.tsx (full read — state management, multi-turn logic, queue, 429 fallback)
- src/presets.ts (full read — 8 presets with prompts, 5 aspect ratios)
- src/components/Sidebar.tsx, EditorArea.tsx, BeforeAfterSlider.tsx, Header.tsx
- src/server/providers/types.ts, index.ts, GoogleAIStudioProvider.ts, VertexAIProvider.ts
- src/utils/composer.ts (full read — 8 Canvas2D preset implementations)
- src/types.ts (ImageEditRequest, ImageVersion, UploadedPhoto types)
- package.json (exact versions for all dependencies)
- tsconfig.json, vite.config.ts
- docs/implementation-plan.md
- git log — 12 commits, 2026-04-27 to 2026-06-11, all by neversettledwanderer